

Understanding Recommendation System Robustness Against Silent Data Corruption: An Empirical Study

Dongning Ma[‡], Xun Jiao^{‡§}, Fred Lin[§], Daniel Moore[§], Sriram Sankar[§]
[‡]Villanova University, [§]Meta

Abstract—Modern deep learning-based recommendation system (DRS) is a dominant workload in industrial data centers. However, with continuous transistor scaling and increasing hardware complexity, silent data corruption (SDC) has become a notable threat to the reliability of data center workloads. Multiple industry hyperscalars have reported the difficulty in addressing SDC due to their “stealthy” nature and elusive manifestation. Given the critical role that DRS plays in maintaining the quality of online services, understanding and enhancing its robustness against SDCs are imperative. To the best of our knowledge, this paper presents the first empirical study on understanding DRS robustness against SDCs. Specifically, we develop *PyTEI*, a PyTorch-based, user-friendly, and highly-efficient error injection framework, based on which we perform large-scale error injection experiments to the parameters of five representative DRS models under three datasets. Experimental results reveal that, the sparsity level of input data and feature affect the robustness of DRS, and in particular, MLP modules inside DRS are especially vulnerable to SDCs. Further, we evaluate the effectiveness of three representative error mitigation methods – algorithm based fault tolerance (ABFT), activation clipping, and selective bit protection (SBP), in enhancing DRS robustness. Experimental results reveal that activation clipping obtains the best result by recovering up to 30% of the degraded DRS performance under SDCs. This study provides valuable insights for industry practitioners in developing robust fault-tolerant strategies for DRS workloads. We open source *PyTEI* at <https://github.com/facebookresearch/PyTEI>.

Index Terms—AI Reliability, Hardware Faults, Recommendation Systems, Robust and Resilience

I. INTRODUCTION

Modern applications including e-commerce, social media, advertising, streaming service and search engine critically rely on recommendation systems to efficiently provide accurate and high quality personalized recommendations [1]. The scale of recommendation systems in production has grown more than an order of magnitude since 2017 with the introduction of deep learning, leading to deep recommendation systems (DRS) demanding high performance systems to ensure QoS such as tail latency and response time [2], [3].

Recently, *silent data corruption* (SDC), referring to a type of “silent” hardware errors that cannot be detected by existing error detection tooling, becomes a significant reliability concern in data centers. Unlike hard failure (e.g., system crash), SDC is especially harmful as it remains undetected and could stealthily corrupt data, which can be catastrophic to end applications [4], [5]. Meta, Google, and Alibaba all report

instances of SDCs in their data centers that require resource-intensive efforts to debug and root cause [4], [6], [7]. For example, Google reported SDCs in their AI systems [8], [9], e.g., Tensor Processing Units (TPUs), which are utilized for running large-scale AI workloads.

While there were various efforts investigating the robustness of AI models such as multi-layer (MLP), convolutional/graph neural networks (CNNs/GNNs) against SDCs [10], [11], there is no existing study on DRS robustness. We find this is a critical gap: first, DRS model is one of the dominating workloads in a data center; for example, in 2019, DRS workload accounted for 79% of the overall AI inference cycles at Meta’s data center [12]. Dedicated hardware accelerators, e.g., MTIA [13], are specifically built for DRS workload. Second, DRS has a notably different model architecture than traditional neural network models: in addition to neural layers, DRS often incorporates large embedding tables accounting for majority of DRS parameters.

In this paper, we present an empirical study to systematically evaluate the impact of SDCs during DRS model inference. The main contributions of the paper are as follows:

- This paper presents the first empirical study of comprehensively evaluating DRS robustness against SDCs. Specifically, we systematically explore multiple model hyper-parameters and input characteristics using a dummy model to identify potential factors that can impact DRS model robustness under bit flip errors. The use of dummy models and synthetic data facilitates the design space exploration process while providing insightful results to guide further experiments.
- We develop a user-friendly, efficient and flexible hardware error injection framework, called *PyTEI*, on top of the widely-used PyTorch framework. Using *PyTEI*, we inject errors into five representative DRS models on three datasets to evaluate their corresponding robustness. We show that the robustness of DRS models can be affected by factors from both architecture and input features, including **DRS-specific features** such as dense/sparse feature ratio and input sparsity of data.
- We discuss or evaluate the effectiveness of three representative error mitigation methods for enhancing DRS robustness, including ABFT, activation clipping and SBP. We find that activation clipping achieves the best results, recovering up to 30% of the degraded performance, which provides valuable insights for industry practitioners to devise strategies in enhancing DRS robustness.

Work done during Xun Jiao’s sabbatical at Meta.

II. RELATED WORKS

SDCs, either permanent hardware faults or soft transient errors, can originate from various sources. Beam testings have revealed that cosmic rays and radiation can induce soft transient errors in GPU memories [14], [15]. Additionally, manufacturing defects and device aging can induce permanent faults which are particularly common in deep learning accelerators with systolic arrays [16], [17]. In addition to faults, hardware errors can also be intentional, e.g., from the trade-off between accuracy and efficiency which is acknowledged as approximate computing [18]. For example, voltage scaling can induce logic or timing errors in memories [19], [20] or computational units [21], which can degrade the deep learning model output quality. Approximate or inexact logic in circuit design can also be applied to functional units such as multipliers for accuracy and power trade-offs [22]. Due to the diverse sources of error, the corresponding error rate can also drastically vary, which motivates us to evaluate across a broad range of bit error rates (BERs) to accurately assess DRS model robustness in this paper.

There are method to mitigate the impact of SDCs in deep learning systems to enhance their robustness. For example, limiting the activation has been explored to improve the resilience of hardware faults for DNNs by confining the output of activation functions within a valid range [23]. For specific data types, critical bits protection is also investigated to mitigate the impact of hardware bit flip errors [24]. Error code correction (ECC) is also commonly used to identify and mitigate the impact of hardware errors, e.g., single bit and double bit error correction using ECC requires about 12.5% redundancy for HBM2 memory on compute-class GPUs [25]. However, ECC may introduce a significant performance penalty to AI processing latency [26]. ABFT has been recently evaluated on the MLP and embedding components of a DRS model on soft errors [27], but it assumes the error correction to be simply re-execution of the operation impacted, which becomes implausible under our error schemes (as detailed in Sec. VIII). In addition, there are other frameworks for fault or resilience analysis such as Thales [28], and Fidelity [29] which consider hardware factors such as flip-flop reuse, but those frameworks still largely focus on single-bit soft and/or transient errors. Such absence motivates us to present results with realistic models and datasets in this paper.

III. BACKGROUND

Deep learning algorithms are introduced into recommendation systems with the objective of providing high quality and accurate personalized contents. A typical task is the click-through-rate (CTR) prediction, where a DRS is developed to estimate the probability of a user's interaction (click, bookmark, purchase, etc.) with a specific item in a given context [30]. Diverse range of DRS have been proposed and implemented. For example, Wide & Deep (WD) from Google considers the joint advantage of neural networks and factorization models (FM) for enhanced performance [31]. Further, DeepFM (DFM) enables the learning of both high-

and low-order feature interactions as well as a sharing strategy of feature embedding which shows increased performance and efficiency [32]. Other experimental and/or commercial architectures such as attentional FM (AFM) [33] and deep cross networks (DCN) [30] have also been introduced for higher performance and efficiency.

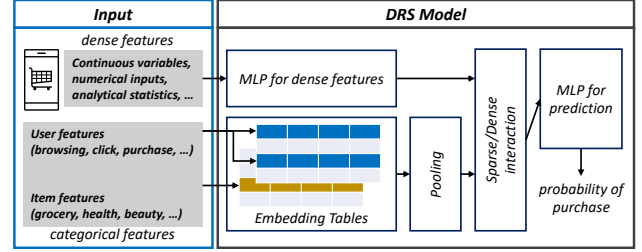


Fig. 1. An example DRS model for purchase prediction.

Without loss of generality, in Fig. 1 we present a typical model of DRS that is trying to predict the purchase probability of an item during a user visit. The input to the DRS model consists of multi-modal features related to the user and the item, where the features can be mainly considered into two groups: the dense features such as the time of day of the visit, and sparse features such as pairing information between users and items. Typically, the dense features are passed to MLPs for feature extraction. The sparse categorical features on the other hand, are handled by embedding tables and converted into latent embeddings. After the sparse/dense interaction, the features are then used to predict the probability of user purchase on the item via another MLP. Such probability can be used on a set of items to select relevant candidates to be recommended to the user.

Processing the dense features using neural networks and processing the sparse features with embedding tables have exhibited drastically different workload characteristics during run-time: the neural networks inside an DRS (rendered by MLPs or attentional layers with matrix operations) are usually compute-intensive, while the embedding processes (rendered by indexing embedding tables) are usually memory-intensive [34].

IV. METHODOLOGY

A. SDC Error Model

In this paper, we focus on SDCs that could corrupt model parameters, which is aligned with previous studies including industrial experiments [10], [11], [15], [27], [35]–[37]. This applies to scenarios where SDCs occur in memory (e.g., DRAM/SRAM where model parameters are stored) due to soft errors, i.e., bit flips. To describe bit flips, we employ the random bit flip model as our error model [10], [11], [15], [27], [35]–[37], where we assume each bit position may get flipped based on a probability referred to as bit error rate (BER). The bit flip errors are also statistically independent from each other, which can be expressed (and implemented) as a Bernoulli distribution: $P(w[\dots] \leftarrow w[\dots] \oplus 1) = p$. Here

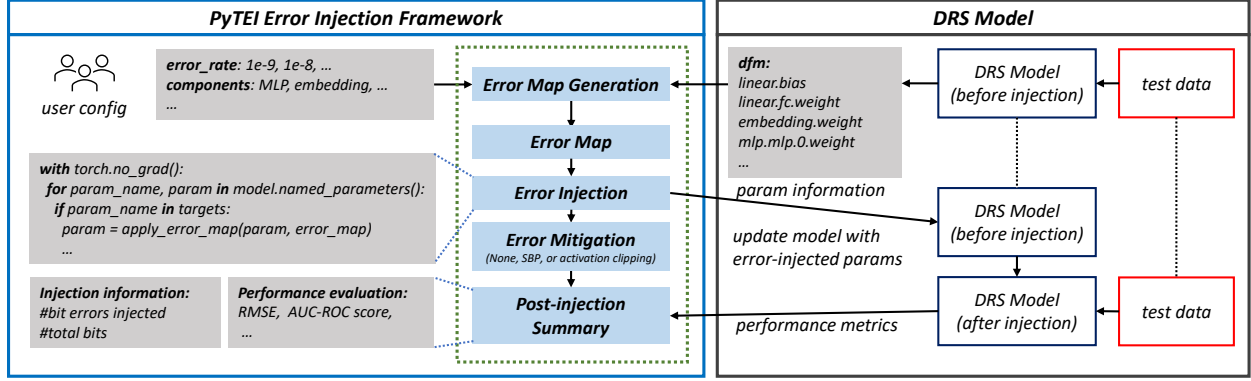


Fig. 2. Overview of injecting bit errors to parameters in DRS models in PyTorch using *PyTEI*.

w refers to the parameter, [...] refers to specific bits in the parameter, $\oplus$ refers to the XOR operation as any bit XOR with 1 will be flipped.

While in practice different bit positions may have varying BERs or there might be a specific error pattern such as burst errors [38], we note this random bit flip model is the most widely-used error model to perform error injection experiments and fault-tolerant research due to its general applicability, including industry efforts [10], [11], [15], [27], [35]–[37]. Without loss of generality, we sweep a wide range of BERs for each bit position (from 10^{-9} to 10^{-2}) in our error injection experiments.

B. Error Injection Framework: *PyTEI*

To enable systematic analysis on DRS robustness, we need efficient error injection and mitigation framework to enable an extensive error injection campaign with reasonable amount of time. We have searched through the available open-sourced error injection frameworks including PyTorchFI [39], Ares [11], BinFI [40], GoldenEye [41] and Fidelity [29]. Nevertheless, these frameworks share certain limitations that render them unsuitable for use in this specific case study. The most challenging obstacle is the efficiency. For example, as floating point tensors in PyTorch do not naturally support bit-level operations (such as bit-wise XOR), these frameworks typically leverage additional packages such as *struct* or *bitstring* for error injection. These packages require frequent conversions between torch tensors and other types of data containers with back-and-forth movements between devices. This accounts for most of the time spent on error injection based on our evaluation, particularly under larger model and higher BERs. Another issue is the tedious setup for environment to use those frameworks. Most of the available frameworks rely on specific environments and/or dockers with numerous additional packages. Some even require users to compile or build CUDA codes by themselves. Moreover, many of them do not provide error mitigation methods or lack of flexible options for hooking in customized error mitigation. As a summary, we need a framework that has the following critical requirements specifically:

- The framework should be able to perform bit-level flips on parameters of PyTorch models to inject emulated hardware errors based on a pre-defined BER. The framework should also provide interface to implement error mitigation methods accordingly.
- The framework should be easily generalized for models with different architectures or components, convenient to setup and require minimal dependency beyond PyTorch.
- The emulation of error injection and mitigation should be considerably fast so that large numbers of different designs can be evaluated efficiently.

To realize such requirements, we develop a framework *PyTEI*, which is an user-friendly, efficient and flexible hardware error injection framework using no other dependencies besides PyTorch. It has the following key features and contributions which essentially enable us to evaluate DRS robustness at large scale efficiently:

- **User-friendly.** *PyTEI* does not strictly require any other dependency other than PyTorch. Note that user may use other packages such as *timeit* and *tqdm* for auxiliary purposes such as tracking the progress and time. Additionally, *PyTEI* also does not require users to compile, build such as other frameworks, nor even use custom environments.
- **Efficient.** *PyTEI* uses PyTorch built-in data type cast (*torch.view()*) in bit flip operations, which significantly (about 100X) accelerates the error injection effort. *PyTEI* can inject bit flip errors at an considerably high BER (10^{-3}) to a model with about 19M parameters within a few seconds using even just a commodity CPU. This also enables *PyTEI* on any PyTorch model as long as they are implemented as *torch.nn.Module* with named parameters.
- **Flexible.** *PyTEI* provides flexible interfaces for customization on both error model and mitigation. *PyTEI* allows user to define their customized error model such as random value error beyond the provided bit flip error. *PyTEI* also implements two mitigation methods for emulation: activation clipping and SBP (introduced in

Sec. V-B) and allows user to customize the mitigation methods.

In Fig. 2, we show a more detailed, step-by-step injection of hardware errors into DRS models using *PyTEI*. Before injection, we use a set of test data to obtain golden outputs (from error-free models) and/or performance scores (e.g. AUC-ROC). In the meanwhile, the user can provide information such as the BER (e.g., $1e-7$), the target (e.g., MLP) and parameters (e.g., weights) for error injection. Based on the BER, injection target and model architecture, we generate an error map to indicate what parameters and which bit positions of the parameter to have error. The model and the error map are then iterated together to update the model parameters. With the model after injection, we use the same test data again to observe outputs which is compared with the golden outputs and/or labels to obtain the output deviation and/or the test score degradation.

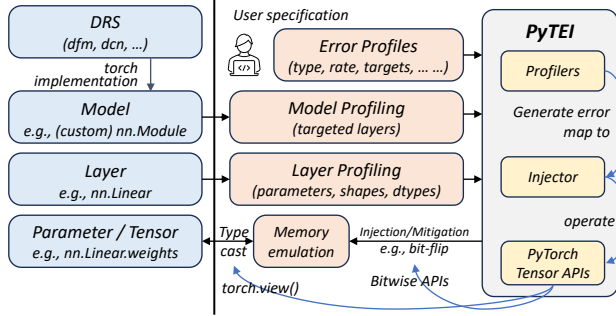


Fig. 3. Top-down view of the *PyTEI* stack.

A top-down view of how *PyTEI* interacts with DRS model and performs error injection and mitigation is illustrated in Fig. 3. *PyTEI* features a profiling functionality that works on the PyTorch implementation of a DRS model by nested iteration. Specifically, *PyTEI* traverses through the model to obtain information of layer modules such as layer type, and then traverses each layer module to obtain parameter attributes such as shapes and data types. Those profiled information of model and model parameters will be used to control error injection under the user specifications such as the error rate and target parameters or layers. The injection functionality of *PyTEI* to emulate hardware errors, i.e., bit-flips on the memory emulation of model parameters (type-cast tensors). Again, as *PyTEI* operates on PyTorch built-in tensor APIs rather than third-party semantics, *PyTEI* can enable (near) real-time error injection.

In Fig. 4 we present an example of the error map generation in *PyTEI*. The error map is generated based on the parameter shape, the bit-width of the data type (which is 32-bit single precision float in this example), as well as a probability representing the error rate. Each value in the error map is a 32-bit integer where the bit positions indicates whether there will be a flip or not for the corresponding bit. The error map along with the number of injected errors are returned.

V. EVALUATION APPROACH

A. Evaluation of DRS Robustness

An overview of experiments in this work to evaluate the robustness of DRS against hardware error is shown in Fig. 5. It features a two-stage evaluation using dummy and realistic models respectively with the error injection framework *PyTEI*. The first stage is to inject errors into dummy models, where the objective is to explore a broad design space to provide insights such as identifying the impact of model hyper-parameters and analyzing which of the components inside a DRS are less robust against hardware errors. The second stage is to inject errors into realistic models with the guidance from the observations using the dummy model. The realistic models are trained and tested with realistic datasets as well. Three error mitigation methods are also discussed and/or evaluated for their effectiveness.

For dummy models, we first provide hyper-parameters such as the depth of MLP layers, the size of MLP hidden layer and the size of the latent embedding to configure the model. The model parameters are initialized uniformly [42] and are not further trained. **The motivation of using dummy models without training is to reduce the effort of model training, which usually takes unrealistic amount of GPU hours (even just by fine-tuning).** Additionally, further experiments using realistic models (which are trained) and data align with the observation using the dummy model as discussed in Sec. VII. Therefore we think it is not plausible or necessary to explicitly train each of the model on the entire huge design space. The dummy model is then injected with error to obtain the error-injected model. Random synthetic data are input to both the models and the RMSE score is calculated based on the outputs of the two models to describe the output deviation after error injection which is used to evaluate the hardware error robustness. The detailed results on dummy models are presented in Sec. VI.

For realistic models, the architecture is implemented based on corresponding literature. Realistic datasets are used to train and test the model to obtain the baseline scores. After error injection, the same test data are input to the error-injected realistic model to obtain the scores after error injection. The score differences are used to evaluate the DRS robustness. The detailed results on dummy models are presented in Sec. VII. The components inside realistic models can be slightly different from that of dummy models due to architectural differences. For example, the attentional mechanisms inside AFM are also considered as MLP due to their similar implementation using fully connected layers and the major computations inside them are also matrix multiplications.

B. Evaluation of Error Mitigation

In this work, we also present preliminary studies on some error mitigation schemes that are in general implemented for other deep learning models: ABFT [27], output clipping [23] and selective bit protection [24]. Analysis and results of evaluation on error mitigation is presented at Sec. VIII.

```

1 @classmethod
2 def _error_map_generate(cls, injectee_shape: tuple, dtype_bitwidth: int, device: torch.device, p: float)
3     ↪ -> torch.Tensor:
4     error_map = (2 * torch.ones((*injectee_shape, dtype_bitwidth), dtype=torch.int, device=device)) **
5     ↪ torch.arange(0, dtype_bitwidth, dtype=torch.int, device=device).flip(dims=(-1,)).expand((*
6     ↪ injectee_shape, dtype_bitwidth))
7     filter = (p * nn.functional.dropout(torch.ones_like(error_map, dtype=torch.float, device=device), 1 -
8     ↪ p)).int()
9     error_count = filter.sum(dim=-1)
10    error_map = (filter * error_map).sum(dim=-1).int()
11    return error_map, error_count

```

Fig. 4. Example of Error Map Generation in PyTEI.

ABFT. ABFT has recently been introduced to DRS for soft error detection [27]. For the general matrix multiply (GEMM) operations like $C = A \times B$, an extra column S_B is appended to B of which each element is the corresponding row-sum, i.e., $S_B[i] = \sum_{j=0}^{n-1} B[i][j]$. Therefore, with the appended column, we can obtain the equality check vector AS_B of which each element **should be** the corresponding row-sum of C . Therefore, if such equality is not hold, an error is detected. The result of this GEMM operation will be discarded and the operation is also repeated. For embedding operations, an additional column C_T is similarly added alongside embedding table T of which each element is the corresponding row-sum as well. Assume indices F are hit during an embedding operation, then the output embedding should be $R = \sum_{f \in F} T[f]$. The same indices are also used to index the elements in the additional column to obtain the equality check value $r = \sum_{f \in F} C_T[f]$ and the value r **should be** equal to the sum of R . Similarly, if such equality is not hold, an error is detected.

Activation Clipping. Clipping the network activations has been investigated to improve the fault tolerance [23], or enable ultra-low precision designs [43]. For example, the ReLU activation function in the model is revised by Eq. (1) [43] where activation will be confined within a range specified by a pre-defined threshold T .

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } 0 \leq x \leq T \\ T & \text{if } x > T \end{cases} \quad (1)$$

Selective Bit Protection. Preliminary studies have illustrated that certain bit positions in specific data types may have dominating effect on the error behavior [24]. For example, in IEEE-754 floating point data format, the value of a 32-bit float number is determined by 3 fields of bits: the sign-bit, the 8-bit exponent field and the 23-bit mantissa field. The most significant bits in the exponent field are observed to have the most critical contribution of the value and is prioritized to have protection from bit errors [24] when it is impossible to protect all the bit positions due to resource limitations. In Sec. VIII, we provide analytical and/or experimental results on those error mitigation schemes and compare their effectiveness and efficiency.

In summary, we aim to explore and answer the following research questions in this case study:

- **RQ1:** How will DRS models generally be impacted by hardware errors? What components inside a DRS model are more vulnerable, or vice versa?
- **RQ2:** How will hyper-parameters such as the hidden layer size of MLPs and the embedding dimensions affect the robustness against hardware errors?
- **RQ3:** Will error mitigation methods for other deep learning models such as ABFT, activation clipping and SBP still be effective for DRS models?

VI. EXPERIMENTS: DUMMY MODELS

A. Dummy Models and Data Generation

For simplicity, we build a dummy DRS model with minimal configuration. The inputs to the model are straightforwardly synthesized, each of which has two vectors representing the dense and sparse features, respectively. To ensure simplicity, the dense features are floating point numbers generated with standard Gaussian distribution, and the sparse features are binary and in Bernoulli given a probability (sparsity). The models consists of a MLP handling the dense features, an embedding table to handle the sparse features and another MLP as the predictor, as shown in Fig. 5. The hyper-parameters of the dummy model and the parameters for generating the dataset which establish the design space to explore according to Tab. I. Specifically, we explore the impact of the depth and hidden layer size of the MLP model, the dimension of embedding for models and the input sparsity and sweep the BER from 10^{-9} to 10^{-2} . We show below that such dummy models with synthetic data have already shown remarkable effectiveness in identifying DRS robustness vulnerability.

TABLE I
HYPER-PARAMETERS OF THE DUMMY MODEL, THE PARAMETERS FOR GENERATING THE INPUT AND THE EVALUATED BERS.

MODEL	MLP DEPTH	1, 2
	MLP HIDDEN LAYER SIZE	64, 128, 256, 512
INPUT	EMBEDDING DIMENSION	64, 128, 256, 512
	DENSE DIMENSION	128
INPUT	SPARSE DIMENSION	8192
	SPARSITY	0.001, 0.01
BIT ERROR RATE		10^{-9} TO 10^{-2}

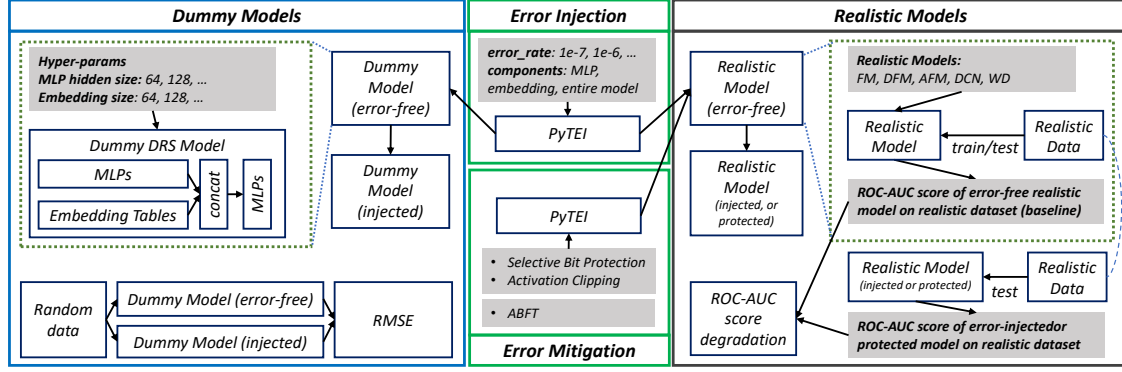


Fig. 5. The methodology of experiments: A two-stage robustness evaluation using the dummy model and the realistic models respectively. Each stage features error injection using *PyTEI*. Three error mitigation methods are also discussed and/or evaluated in the stage of realistic models.

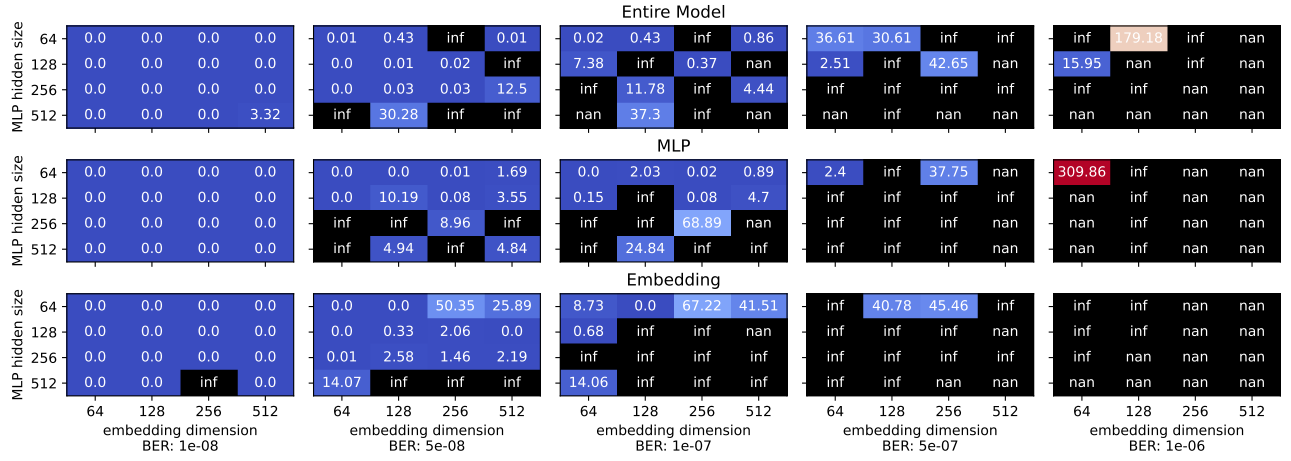


Fig. 6. Root mean square error (RMSE) between outputs of error-free models and outputs from error-injected models under different BERs, MLP hidden layer size and embedding dimensions. Errors are injected into different components inside the model as outlined by the title of each row.

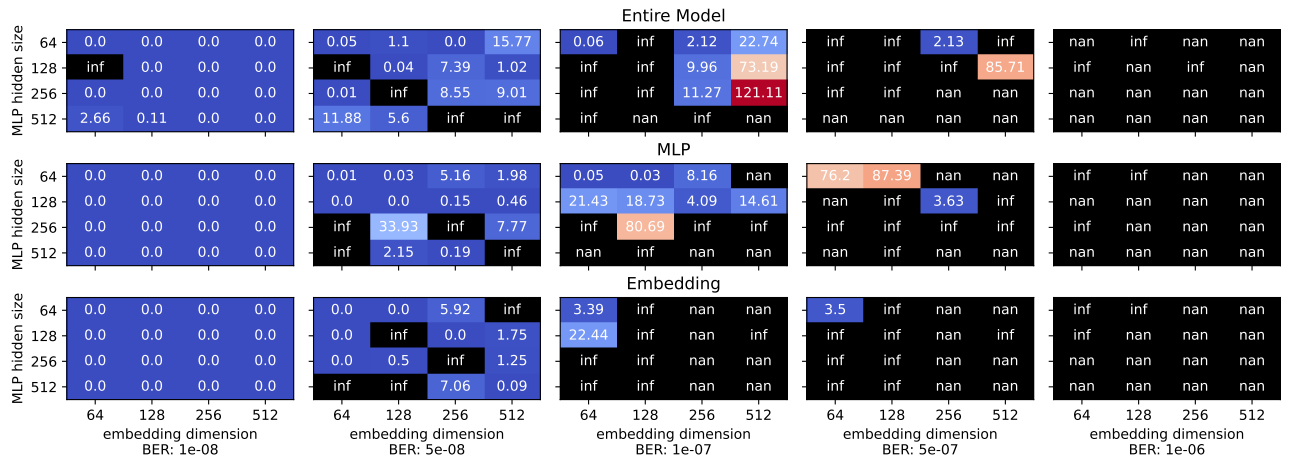


Fig. 7. Root mean square error (RMSE) between outputs of error-free models and outputs from error-injected models under different BERs, MLP hidden layer size and embedding dimensions when the number of hidden layers inside MLP is increased from 1 to 2. Errors are injected into different components inside the model as outlined by the title of each row.

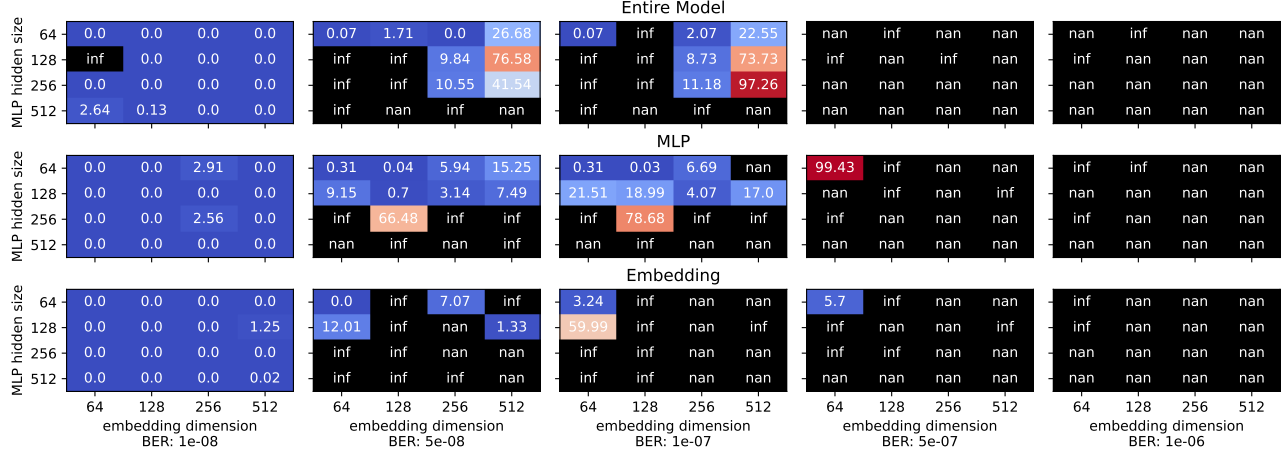


Fig. 8. Root mean square error (RMSE) between outputs of error-free models and outputs from error-injected models under different **BERs**, **MLP hidden layer size** and **embedding dimensions** when the number of hidden layers inside MLP is increased from 1 to 2 and the input sparsity is increased by 10X to 0.01. Errors are injected into different components inside the model as outlined by the title of each row.

B. Robustness Analysis of Dummy Models

As briefly described in Sec. V-A and Fig. 5, for dummy models, we use the RMSE between the outputs from error-free and error-injected models to evaluate the deviation and then characterize the impact of hardware errors. In Fig. 6, Fig. 7, and Fig. 8, we show the RMSE metrics of the all the design points explored. We omit the RMSEs for BERs below 10^{-8} and above 10^{-6} because they are either in all zeros or all *inf* or *nan* numbers. According to our observation, there are possibilities that the bit errors happen on critical positions within a number such as the exponent field in the IEEE-754 32-bit floating point number and result in extremely large deviations (e.g., flipping the 2nd bit of the number 0.625 will get 2.13×10^{38}) which, after network propagation, can exceed the range of floats and become *inf* or *nan*. Therefore, if there are more than 1 such invalid numbers of the output, we mark the RMSE as *inf* or *nan* accordingly in the figure. On the other hand, we mark any RMSE less than 0.005 as 0.0 for presenting in the figure, indicating there is no noticeable errors observed. For example, the 0.0 at the upper left slot of the row “Entire Model” indicates that when both the **MLP hidden size** and the **embedding dimension** are 64, injecting errors to the weights on the **entire model** does not induce in any noticeable error on the output.

We can make several observations from the figure. Overall for the figures from Fig. 6 to Fig. 8, when the BER increases the RMSE metric will also increase accordingly. This is intuitive and aligns with the observations of other machine learning models in general, since more errors injected into the model will likely translate into higher output deviations. However, we do observe that tuning the hyper-parameters of different components inside a DRS can uncover the different characteristics of robustness. For example, although by increasing the MLP hidden size (from top to down in each sub-figure) and the embedding dimension (from left to right

in each sub-figure) can both lead to increasing error, MLP hidden size seems to induce more impact by having higher RMSE, or more invalid numbers.

Based on the results we infer that such difference between MLP and embedding originates from their corresponding architecture. For MLP, even with drop-out schemes, each weight as a parameter will more or less contribute when calculating prediction results with the forward pass. Therefore, when increasing the hidden layer size of MLP, the outputs also become more influenced. However, as to embedding, for each sample only a limited number of entries in the embedding table are indexed due to the sparsity of inputs, therefore it is of a high chance that the indexed entries are not affected by the error. Thus, increasing the embedding dimension does not exacerbate the output error as significant as MLP.

Impact of MLP depth. In Fig. 7, we show the results when increasing the depth of the MLPs in the DRS model by adding another hidden layer. It can be observed that the added hidden layer deteriorates the output quality regardless of whether the error injection is into MLP, embedding or the entire model. According to our analysis, the reason behind of the increasing error is two-fold: first, an additional hidden layer in MLP indicates more parameters in general. Thus, with the same BER the absolute number of bit errors is also higher. Second, if the bit error in the first hidden layer result in an erroneous value, it can be propagated to the second layer and cause more erroneous values due to the fully connection.

Impact of input sparsity. In Fig. 8, we further change the sparsity of the sparse vectors in the input set by 10X from 0.001 to 0.01. By comparing the last row “Embedding” in Fig. 8 and Fig. 7, we can observe that since more entries are indexed within the embedding table, more errors are incorporated in the latent embeddings. Then the impact from error-injected embedding table will subsequently become more pronounced, which is presented by having higher RMSE

and/or more invalid numbers.

As a summary, we can have the following insights: First, similar to other machine learning models, DRS model suffers a similar trend that when the BER increases, the output quality will degrade in general. However, the hyper-parameters of different components inside a DRS can pose different impact on the robustness. This is resulted from the inherent architectural differences of those components. When the BER increases to around 10^{-6} , nearly all the results witness invalid values in their outputs. Therefore, we can infer that for the experiments on realistic model and data, the model performance will experience noticeable degradation for BERs above 10^{-6} .

VII. EXPERIMENTS: REALISTIC MODELS

A. Realistic Models and Datasets

We target at 5 widely acknowledged recommendation systems: Factorization Machine (FM) [44], Deep Factorization Machine (DFM) [32], Attentional Factorization Machine (AFM) [33], Deep Cross Network (DCN) [30], and Wide and Deep (WD) [31]. The models are implemented from *torchfm*². The details about the evaluated models are listed in Tab. II. As it is very difficult to find pre-trained PyTorch models for those DRS models since different models use different frameworks for implementations and datasets for training, we train all the models afresh using the default parameters in *torchfm*.

TABLE II
MODELS, DATASETS AND BASELINE SCORES

MODEL	DATASET	#PARAM.	AUC-ROC
FM	MOVIELENS-1M	171K	0.813
	MOVIELENS-20M	4.59M	0.838
	CRITEO	18.5M	0.779
DFM	MOVIELENS-1M	170K	0.842
	MOVIELENS-20M	4.59M	0.838
	CRITEO	18.5M	0.785
AFM	MOVIELENS-1M	170K	0.862
	MOVIELENS-20M	4.58M	0.827
	CRITEO	18.5M	0.801
DCN	MOVIELENS-1M	161K	0.824
	MOVIELENS-20M	4.32M	0.818
	CRITEO	17.4M	0.799
WD	MOVIELENS-1M	170K	0.828
	MOVIELENS-20M	4.59M	0.815
	CRITEO	18.5M	0.791

We use 3 benchmark datasets for CTR prediction: Movielens-1M³, Movielens-20M and Criteo⁴ DAC dataset. The Movielens-1M dataset contains 1 million ratings from 6,000 users on 4,000 movies and the Movielens-20M dataset contains 20 million ratings and 465,000 tag applications applied to 27,000 movies by 138,000 users. The Criteo dataset

contains the click records of 45 million users with 13 continuous and 26 categorical features. The datasets are randomly split by 8:1:1 for training, validation and testing.

We use AUC-ROC as our main quality metric for consistency on comparison since it is used mostly across the papers of the DRS models. The baseline AUC-ROCs reported in Tab. II mostly align with what the original research papers present with minor differences.

B. Robustness Analysis of Realistic Models

In Fig. 9, Fig. 10, and Fig. 11, we present the model performance of the evaluated realistic DRS models under different BERs from 10^{-9} to 10^{-2} . The line shows the average of the AUC-ROC scores or accuracy while the shade represents their range observed from 10 individual experiments. We can first observe that the general trend is consistent with what we have observed from experiments on dummy models: with higher BER, the model performance suffers from higher quality degradation. Moreover, for most of the models, the scores exhibit noticeable quality loss after the BER of 10^{-6} , which also aligns with our inference using dummy models. However, severe score drops start from 10^{-5} or 10^{-4} and most of the models becomes completely collapsed with BER higher than 10^{-3} .

By using dummy models we observe that different components inside the DRS model have different robustness characteristics. We also have similar observations from realistic models: the embedding tables are more robust against error as the significant error drops start after BER 10^{-3} as shown in Fig. 10. The range of scores of 10 runs is also more consolidated near the average by having smaller shading areas. Moreover, models with more MLP architectures such as DCN, DFM and WD are less robust against hardware errors compared with embedding-dominant architectures such as FM (which only has MLP for the final predictor), based on Fig. 11.

We have also observed that different datasets can also impact the model robustness. For dummy models, we use randomly synthesized data with fixed dimensions of dense and sparse features as well as the sparsity. However, it can be observed that models developed for the Criteo dataset are less robust against errors, as the BER where they exhibit noticeable score degradation is lower than that of the two MovieLens datasets. This can result from dataset-specific attributions, for example the models developed for Criteo have in general higher amount of MLP architectures compared with the rest datasets. The Criteo dataset has less sparsity which also influences the robustness of DRS model as analyzed using dummy models.

As a summary, we can have the following insights: In general, the robustness of DRS against hardware error intuitively follows the trend of other deep learning models, i.e., the score gradually decreases when BER increases. However, robustness drastically (up to 2 orders of magnitude) vary between different models and datasets. Such huge variation, based on our analysis and observation, originates from two major aspects: the architectural differences and the characteristics of

²<https://github.com/rixwew/pytorch-fm>

³<https://grouplens.org/datasets/movielens>

⁴<https://ailab.criteo.com/ressources>

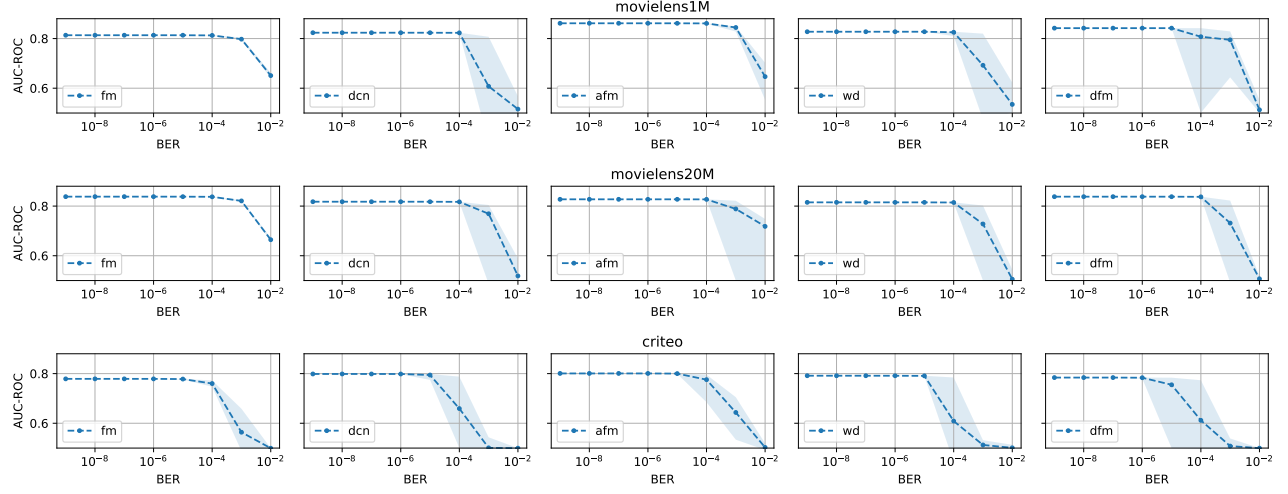


Fig. 9. Performance of recommendation systems under different bit error rates. Errors are injected into the weights in *both embedding tables and neural networks* in each model. Results are averaged from 10 different runs of error injection with shaded areas covering the range of maximum and minimum scores observed.

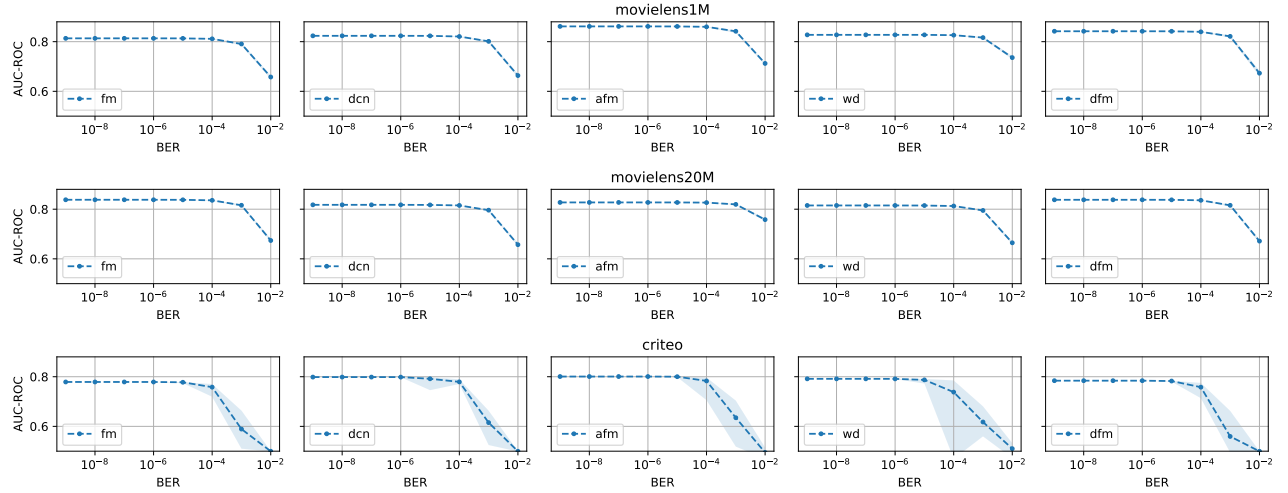


Fig. 10. Performance of recommendation systems under different bit error rates. Errors are injected into the weights in the *embedding tables*. Results are averaged from 10 different runs of error injection with shaded areas covering the range of maximum and minimum scores observed.

input data. DRS models with more MLP architectures, higher amount of dense input features and less sparsity in sparse input features are likely to be more susceptible against hardware errors and requires more intensive error mitigation.

VIII. EXPERIMENTS: ERROR MITIGATION

ABFT. ABFT is proved to be effective to detect soft errors in DRS of having 99% effectiveness and 10% false positive of bit flip error detection with overhead below 26% on various DRS architectures [27]. However, most of such error correction methods are capable of single-error correcting and double-error detecting (SEC-DED), and simply re-execute when an error is detected but not correctable. However, when

we attempt to implement ABFT, particularly when the BER is higher than 10^{-4} , almost all the matrices for parameters contain multiple errors which always force re-execution of operations. Therefore, we conclude that such error-correction-code based methods work for intermittent, low-rate soft and transient errors caused by, e.g., cosmic rays and radiation effects yet are infeasible for scenarios with higher BERs which can come from logic and data-path errors, or voltage and frequency scaling.

Activation Clipping and SBP. For activation clipping we manually clip the activations inside the range of $[-6, 6]$ (inspired by ReLU6 [45]). For SBP, we protect the sign and exponent bits. Although this work, which aims to provide

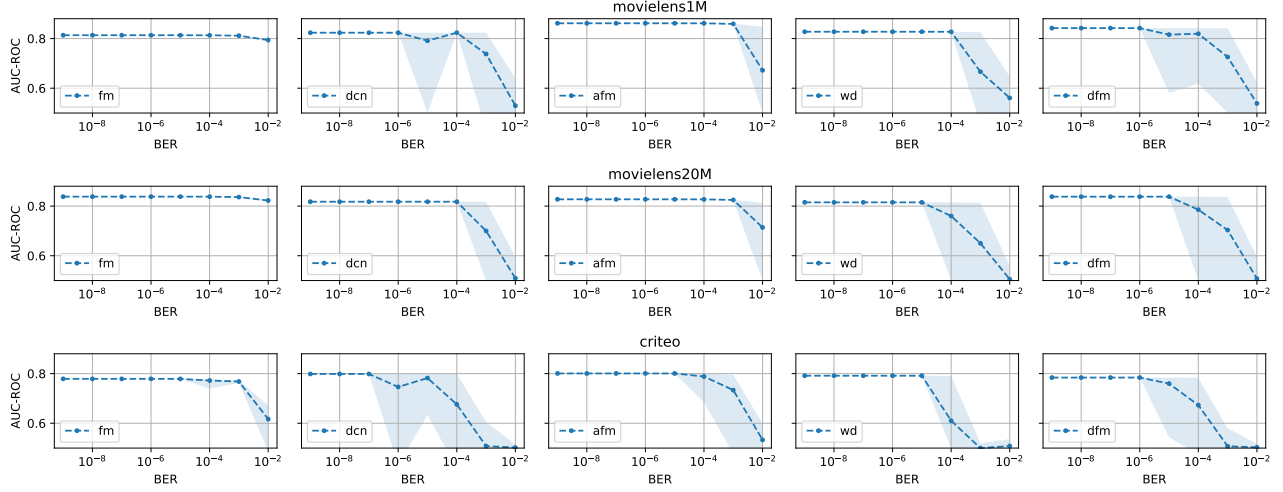


Fig. 11. Performance of recommendation systems under different bit error rates. Errors are injected into the weights in the *neural networks* in each model. Results are averaged from 10 different runs of error injection with shaded areas covering the range of maximum and minimum scores observed.

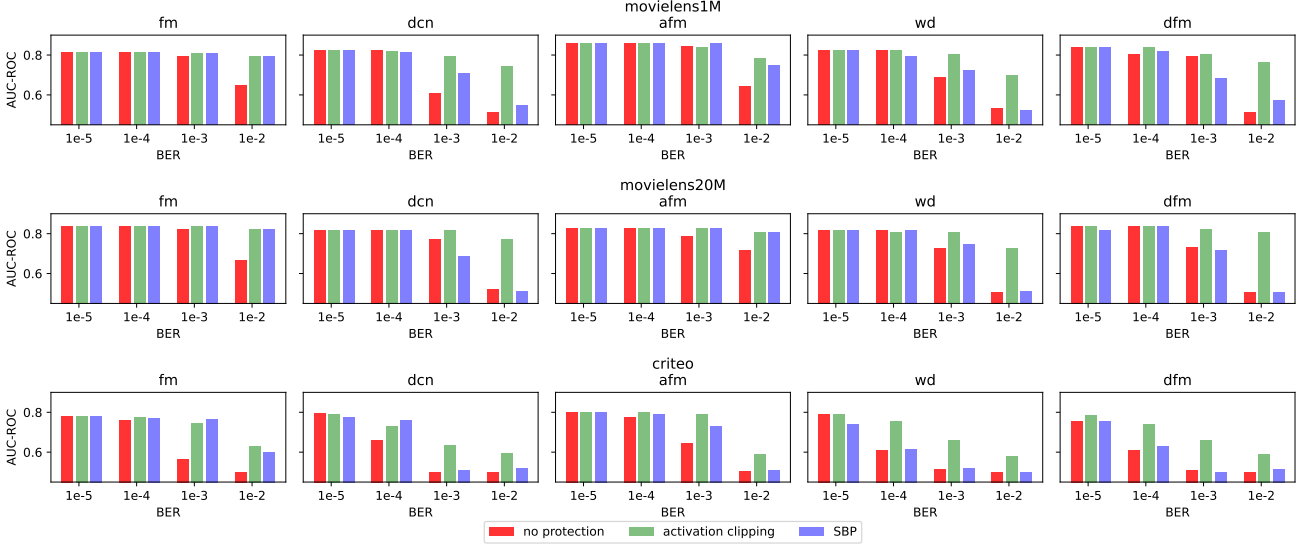


Fig. 12. Comparison between no protection, activation clipping, and selective bit protection under different BERs. Results are averaged from 10 individual runs.

primary evaluation on the error mitigation methods for DRS, does not focus on exploring the optimal parameters for activation clipping or the bit positions to protect for specific DRS models, we provide functionality for users to customize those parameters in *PyTEI*. We present the results of error mitigation using activation clipping and SBP in Fig. 12. The BERs start from 10^{-5} since any lower BER does not incur significant degradation in score. We can observe that mostly, both of the error mitigation schemes are effective to regain some model performance across all the DRS models and all the datasets by 5% to 30%.

The effectiveness of error mitigation also varies across different datasets. Generally, models that show stronger robust-

ness (FM and AFM) as discussed in Sec. VII also experience better recovery. Those observations align with our analysis that the MLP components inside DRS models, when having error, contribute to most of the performance degradation. Activation clipping is more effective on regaining the score loss for models that have deep MLP architectures such as DCN, WD and DFM, while SBP functions more effectively for embedding table dominated architectures. Efficiency-wise, protecting selective bit positions require dedicated mechanisms or hardware while clipping activation of MLP layers are less resource-limiting since it can be simply implemented by modifying the activation functions.

IX. DISCUSSION

A. Main Observations

In this section, we discuss the main observations of this case study to answer the proposed research questions:

RQ1: How will DRS models generally be impacted by hardware errors? What components inside a DRS model are more vulnerable to hardware errors, or vice versa?

A1: Similar to other deep learning models, DRS models show certain robustness but can also be negatively impacted by hardware errors, depending on the error locations and rates. Different components inside a DRS exhibit different hardware error robustness due to the architectural differences and different roles they play in DRS. According to observations from Sec. VI and Sec. V-A, MLP architectures are usually considered less robust while embedding tables are more robust against hardware errors. Therefore, we recommend future system designers to prioritize MLPs for protection to mitigate the impact of hardware errors.

RQ2: How will hyper-parameters such as the hidden layer size of MLPs and the embedding dimensions affect the robustness against hardware errors?

A2: Tuning the hyper-parameters will influence the robustness of DRS models. Similarly, different components inside DRS also exhibit different sensitivity on hyper-parameter tuning. In general, by increasing the hidden layer size of MLPs in DRS, the robustness suffers from more severe degradation compared with increasing the embedding dimension. Additionally, dense/sparse feature ratio and input sparsity are the unique robustness characteristics of DRS compared with other deep learning models, which are originated from the architectural difference between MLP and embedding components, as analyzed in Sec. VI.

RQ3: Will error mitigation methods for other deep learning models such as ABFT, activation clipping and SBP still be effective for DRS models?

A3: Conventional ABFT is implausible to enhance the robustness of DRS under our error scenario due to their theoretical limitations. Activation clipping and SBP are effective for the evaluated DRS models to recover degraded score. However, activation clipping is observed to be more effective than SBP since it specifically protects the MLP components inside the DRS model which are less robust against hardware errors based on our analysis. Activation clipping also requires less overhead compared with SBP as it can be implemented in algorithm level directly.

B. Limitations and Future Works

Inference vs. Training. In this empirical study, we focus on DRS robustness during inference because the impact of SDCs could directly impact user experience. However, we note that SDCs can also impact AI training. For example, SDCs has impacted Llama 3 training in Meta [46]. SDCs in training often cause different outcomes than inference, e.g., gradient explosion, loss spike, or non-convergence of training process [8]. Our future work will extend *PyTEI* to support model training, and investigate model robustness during training phase.

Error Model. In this paper, we assume a random bit flip error model for error injection experiments for its wide usage and alignment with existing studies. We acknowledge that there are different error models across different hardware platforms and the error model used in this paper do not fully represent all possible error models. For example, burst errors can upset one or more adjacent bits [38]. There are also additional type of hardware faults such as permanent stuck-at faults and multi-bit errors [47]. Thus, our future effort will focus on evaluating DRS behavior under different error patterns and types.

General Applicability. *PyTEI* is originally developed for DRS, however it can further handle error injection and mitigation to almost any PyTorch model implemented as *nn.Module* such as MLP/CNN/RNN. Recently, generative models, large language models (LLM) and their corresponding services become another major AI workload in data centers. Thus, using this paper as a starting point, we will also endeavor to extend the use of *PyTEI* to these models and broaden the scope of our study.

X. CONCLUSION

In this case study, we systematically analyze the robustness of DRS with regard to silent data corruptions. We develop and then open-source *PyTEI*, a user-friendly, efficient and flexible framework for error injection. We identify that the MLP components inside DRS models are particularly sensitive to hardware errors, while other major factors such as the input sparsity can also affect DRS robustness. We also evaluate three error mitigation methods including ABFT, activation clipping and SBP. Activation clipping, based on our observation, is the most promising error mitigation method which can regain up to 30% of the model performance with minimal overhead. This study provides valuable insights in developing fault-tolerant strategies for DRS workloads.

REFERENCES

- [1] Z. Zhao, L. Hong, L. Wei, *et al.*, "Recommending what video to watch next: A multitask ranking system," in *Proceedings of the 13th ACM Conference on Recommender Systems*, 2019, pp. 43–51.
- [2] W. Zhao, J. Zhang, D. Xie, Y. Qian, R. Jia, and P. Li, "Aibox: Ctr prediction model training on a single node," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 2019, pp. 319–328.
- [3] W. Zhao, D. Xie, R. Jia, *et al.*, "Distributed hierarchical gpu parameter server for massive scale deep learning ads systems," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 412–428, 2020.
- [4] S. Wang, G. Zhang, J. Wei, Y. Wang, J. Wu, and Q. Luo, "Understanding silent data corruptions in a large production cpu population," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 216–230.
- [5] H. D. Dixit *et al.*, "Detecting silent data corruptions in the wild," *arXiv preprint arXiv:2203.08989*, 2022.
- [6] H. D. Dixit, S. Pendharkar, M. Beadon, *et al.*, "Silent data corruptions at scale," *arXiv preprint arXiv:2102.11245*, 2021.
- [7] P. H. Hochschild, P. Turner, J. C. Mogul, *et al.*, "Cores that don't count," in *Proceedings of the Workshop on Hot Topics in Operating Systems*, 2021, pp. 9–16.
- [8] Y. He, M. Hutton, S. Chan, *et al.*, "Understanding and mitigating hardware failures in deep learning training systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.

- [9] J. Dean and A. Vahdat, "Exciting directions for ml models and the implications for computing hardware," in *Keynote 1, Hot Chips 2023*, 2023.
- [10] G. Li, S. K. S. Hari, M. Sullivan, *et al.*, "Understanding error propagation in deep learning neural network (dnn) accelerators and applications," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2017, pp. 1–12.
- [11] B. Reagen, U. Gupta, L. Pentecost, *et al.*, "Ares: A framework for quantifying the resilience of deep neural networks," in *Proceedings of the 55th Annual Design Automation Conference*, 2018, pp. 1–6.
- [12] S. Hsia, U. Gupta, B. Acun, *et al.*, "Mp-rec: Hardware-software co-design to enable multi-path recommendation," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 449–465.
- [13] A. Firoozshahian, J. Coburn, R. Levenstein, *et al.*, "Mtia: First generation silicon targeting meta's recommendation systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.
- [14] D. Tiwari, S. Gupta, J. Rogers, *et al.*, "Understanding gpu errors on large-scale hpc systems and the implications for system design and operation," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, IEEE, 2015, pp. 331–342.
- [15] M. B. Sullivan, N. Saxena, M. O'Connor, *et al.*, "Characterizing and mitigating soft errors in gpu dram," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 641–653.
- [16] M. A. Hanif and M. Shafique, "Dependable deep learning: Towards cost-efficient resilience of deep neural network accelerators against soft errors and permanent faults," in *2020 IEEE 26th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, IEEE, 2020, pp. 1–4.
- [17] Y. Zhao, K. Wang, and A. Louri, "Fsa: An efficient fault-tolerant systolic array-based dnn accelerator architecture," in *2022 IEEE 40th International Conference on Computer Design (ICCD)*, IEEE, 2022, pp. 545–552.
- [18] V. Leon, M. A. Hanif, G. Armeniakos, *et al.*, "Approximate computing survey, part ii: Application-specific & architectural approximation techniques and applications," *arXiv preprint arXiv:2307.11128*, 2023.
- [19] I. E. Yüksel, B. Salami, O. Ergin, O. S. Unsal, and A. C. Kestelman, "Mors: An approximate fault modeling framework for reduced-voltage srams," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 6, pp. 1663–1673, 2021.
- [20] A. G. Yağlıkcı, H. Luo, G. F. De Oliveira, *et al.*, "Understanding rowhammer under reduced wordline voltage: An experimental study using real dram devices," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, IEEE, 2022, pp. 475–487.
- [21] X. Jiao, D. Ma, W. Chang, and Y. Jiang, "Levax: An input-aware learning-based error model of voltage-scaled functional units," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 5032–5041, 2020.
- [22] V. Mrazek, S. S. Sarwar, L. Sekanina, Z. Vasicek, and K. Roy, "Design of power-efficient approximate multipliers for approximate artificial neural networks," in *2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, ACM, 2016, pp. 1–7.
- [23] L.-H. Hoang, M. A. Hanif, and M. Shafique, "Ft-clipact: Resilience analysis of deep neural networks and improving their fault tolerance using clipped activation," in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, IEEE, 2020, pp. 1241–1246.
- [24] Y. Chen, M. F. Reza, and A. Louri, "Dec-noc: An approximate framework based on dynamic error control with applications to energy-efficient nocs," in *2018 IEEE 36th International Conference on Computer Design (ICCD)*, IEEE, 2018, pp. 480–487.
- [25] H.-M. Chen, S.-Y. Lee, T. Mudge, C.-J. Wu, and C. Chakrabarti, "Configurable-ecc: Architecting a flexible ecc scheme to support different sized accesses in high bandwidth memory systems," *IEEE Transactions on Computers*, vol. 68, no. 5, pp. 646–659, 2018.
- [26] N. P. Joppi, D. H. Yoon, M. Ashcraft, *et al.*, "Ten lessons from three generations shaped google's tpuv4i: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, IEEE, 2021, pp. 1–14.
- [27] S. Li, J. Huang, P. T. P. Tang, *et al.*, "Efficient soft-error detection for low-precision deep learning recommendation models," in *2022 IEEE International Conference on Big Data (Big Data)*, IEEE, 2022, pp. 1556–1563.
- [28] A. Tyagi, Y. Gan, S. Liu, B. Yu, P. Whatmough, and Y. Zhu, "Thales: Formulating and estimating architectural vulnerability factors for dnn accelerators," *arXiv preprint arXiv:2212.02649*, 2022.
- [29] Y. He, P. Balaprakash, and Y. Li, "Fidelity: Efficient resilience analysis framework for deep learning accelerators," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, IEEE, 2020, pp. 270–281.
- [30] R. Wang, B. Fu, G. Fu, and M. Wang, "Deep & cross network for ad click predictions," in *Proceedings of the ADKDD'17*, 2017, pp. 1–7.
- [31] H.-T. Cheng, L. Koc, J. Harmsen, *et al.*, "Wide & deep learning for recommender systems," in *Proceedings of the 1st workshop on deep learning for recommender systems*, 2016, pp. 7–10.
- [32] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, "Deepfm: A factorization-machine based neural network for ctr prediction," *arXiv preprint arXiv:1703.04247*, 2017.
- [33] J. Xiao, H. Ye, X. He, H. Zhang, F. Wu, and T.-S. Chua, "Attentional factorization machines: Learning the weight of feature interactions via attention networks," *arXiv preprint arXiv:1708.04617*, 2017.
- [34] S. Hsia, U. Gupta, M. Wilkening, C.-J. Wu, G.-Y. Wei, and D. Brooks, "Cross-stack workload characterization of deep recommendation systems," in *2020 IEEE International Symposium on Workload Characterization (IISWC)*, IEEE, 2020, pp. 157–168.
- [35] B. Reagen, P. Whatmough, R. Adolf, *et al.*, "Minerva: Enabling low-power, highly-accurate deep neural network accelerators," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 267–278, 2016.
- [36] S. Kim, P. Howe, T. Moreau, A. Alaghi, L. Ceze, and V. Sathe, "Matic: Learning around errors for efficient low-voltage neural network accelerators," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, IEEE, 2018, pp. 1–6.
- [37] B. Sangchoolie, K. Pattabiraman, and J. Karlsson, "One bit is (not) enough: An empirical study of the impact of single and multiple bit-flip errors," in *2017 47th annual IEEE/IFIP international conference on dependable systems and networks (DSN)*, IEEE, 2017, pp. 97–108.
- [38] A. Radonjic, "Integer codes correcting single errors and detecting burst errors within a byte," *IEEE Transactions on Device and Materials Reliability*, vol. 20, no. 4, pp. 748–753, 2020.
- [39] A. Mahmoud, N. Aggarwal, A. Nobbe, *et al.*, "Pytorchfi: A runtime perturbation tool for dnns," in *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, IEEE, 2020, pp. 25–31.
- [40] Z. Chen, G. Li, K. Pattabiraman, and N. DeBardeleben, "Binfi: An efficient fault injector for safety-critical machine learning systems," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2019, pp. 1–23.
- [41] A. Mahmoud, T. Tambe, T. Aloui, D. Brooks, and G.-Y. Wei, "Goldeneeye: A platform for evaluating emerging numerical data formats in dnn accelerators," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, IEEE, 2022, pp. 206–214.
- [42] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1026–1034.
- [43] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, "Pact: Parameterized clipping activation for quantized neural networks," *arXiv preprint arXiv:1805.06085*, 2018.
- [44] S. Rendle, "Factorization machines," in *2010 IEEE International conference on data mining*, IEEE, 2010, pp. 995–1000.
- [45] A. G. Howard, M. Zhu, B. Chen, *et al.*, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [46] A. Grattafiori, A. Dubey, A. Jauhri, *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [47] V. Sridharan and D. Liberty, "A study of dram failures in the field," in *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, IEEE, 2012, pp. 1–11.